Practice
Query
Grow

Same
Concepts
Bigger
Opportunities !

50 MOST ASKED SQL QUERIES

SELECT
LEARN
PRACTICE
MASTER

Good
Queries
Better
Developers !

From
Basics
to
Interview
Ready !

Write
Query
Solve
Grow

Handpicked Queries for Real-World & Interviews

Learn • Practice • Apply • Get Hired

WORKS ACROSS ALL MAJOR SQL DATABASES

MySQL | PostgreSQL | SQL Server | Oracle | SQLite | ...

COVERS KEY CONCEPTS

- [x] SELECT, WHERE, ORDER BY
- [x] GROUP BY, HAVING
- [x] JOINS (INNER, LEFT, RIGHT, FULL)
- [x] SUBQUERIES
- [x] CTEs & CASE
- [x] WINDOW FUNCTIONS
- [x] DATE & STRING QUERIES
- [x] ADVANCED QUERIES
- [x] REAL-WORLD SCENARIOS

SQL

```sql
SELECT
    skills,
    opportunities
FROM
    practice
WHERE
    consistency = 'true'
ORDER BY
    success DESC;
```

" Small
Queries
Big
Careers ! "

Crack Interviews | Build Confidence | Improve Problem Solving | Real-World Use Cases | Better Opportunities

Consistent
Practice

@codedsoul_05

Good
Queries

SQL
Step by Step
Stronger
Foundations !

1. Basic SELECT + Filtering (Q1 – Q5)

Small Queries Big Concepts ! ☺

• Q1. Get all records from a table

Problem :
Write a query to fetch all records from the employees table.

```
SELECT *
FROM employees;
```

* is used to select all columns

Result (sample) :

id	name	department	salary
1	John	IT	60000
2	Emma	HR	55000
3	Mike	Finance	70000

• Q2. Select specific columns

Problem :
Write a query to fetch only the name and salary of all employees.

```
SELECT name, salary
FROM employees;
```

Use specific columns instead of SELECT * for better performance

Result (sample) :

name	salary
John	60000
Emma	55000
Mike	70000

• Q3. Filter records using WHERE

Problem :
Write a query to get all employees from the IT department.

```
SELECT *
FROM employees
WHERE department = 'IT';
```

Use WHERE to filter rows based on a condition

Result (sample) :

id	name	department	salary
1	John	IT	60000
4	Sarah	IT	65000

• Q4. Filter using multiple conditions

Problem :
Write a query to get employees from the IT department with salary greater than 60000.

```
SELECT *
FROM employees
WHERE department = 'IT'
  AND salary > 60000;
```

Use AND, OR to combine multiple conditions

Result (sample) :

id	name	department	salary
4	Sarah	IT	65000
5	David	IT	80000

• Q5. Filter records using LIKE

Problem :
Write a query to find employees whose name starts with 'J'.

```
SELECT *
FROM employees
WHERE name LIKE 'J%';
```

LIKE is used for pattern matching

Result (sample) :

id	name	department	salary
1	John	IT	60000
8	Jessica	HR	58000

Key Takeaways

- ✓ SELECT is used to fetch data
- ✓ Use specific columns instead of *
- ✓ WHERE filters records
- ✓ Use AND, OR for multiple conditions
- ✓ LIKE helps with pattern matching

Practice SQL
Build Better Skills ! ☺

@codedsoul_05

SQL
Step by Step
Stronger
Foundations !

2. Sorting + Aggregation (Q6 – Q10)

Practice
Write
Run
Get Better ! ☺

• Q6. Sort records in ascending order

Problem :
Get all employees sorted by salary in ascending order (lowest to highest).

```
SELECT *
FROM employees
ORDER BY salary ASC;
```

Use ASC for ascending order (default).

Result (sample) :

id	name	department	salary
4	Sarah	HR	45000
2	Emma	HR	55000
1	John	IT	60000
3	Mike	Finance	70000

• Q7. Sort records in descending order

Problem :
Get all employees sorted by salary in descending order (highest to lowest).

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Use DESC for descending order.

Result (sample) :

id	name	department	salary
3	Mike	Finance	70000
5	David	IT	68000
1	John	IT	60000
2	Emma	HR	55000

• Q8. Get the total number of records

Problem :
Find the total number of employees in the table.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

COUNT(*) counts all rows.

Result (sample) :

total_employees
5

• Q9. Find the maximum and minimum salary

Problem :
Get the highest and lowest salary from the employees table.

```
SELECT MAX(salary) AS max_salary,
       MIN(salary) AS min_salary
FROM employees;
```

Use Aggregate functions like MAX() and MIN().

Result (sample) :

max_salary	min_salary
70000	45000

• Q10. Get the average salary by department

Problem :
Find the average salary of employees in each department.

```
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Use AVG() with GROUP BY to get department wise results.

Result (sample) :

department	avg_salary
Finance	70000
HR	50000
IT	67667

Key Takeaways

- ✓ Use ORDER BY to sort results (ASC/DESC)
- ✓ Aggregate functions: COUNT(), MAX(), MIN() AVG()
- ✓ Use GROUP BY for grouped results
- ✓ Aggregation helps in getting summary insights
- ✓ These queries are commonly asked in interviews

Keep Practicing
Keep Growing ! ☺

@codedsoul_05

SQL
Step by Step
Stronger
Foundations !

3. GROUP BY + HAVING (Q11 – Q15)

Group
Analyze
Filter
Master SQL !

@codedsoul_05

• Q11. Count employees in each department

Problem :
Write a query to find the number of employees in each department.

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department;
```

GROUP BY groups rows with same value !

Result (sample) :

department	total_emp
HR	3
IT	4
Finance	2
Marketing	1

• Q12. Find total salary paid per department

Problem :
Write a query to find the total salary in each department.

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

Use SUM() to calculate total values per group !

Result (sample) :

department	total_salary
HR	175000
IT	240000
Finance	130000
Marketing	70000

• Q13. Find average salary per department

Problem :
Write a query to find the average salary in each department.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Use AVG() for average value per group !

Result (sample) :

department	avg_salary
HR	58333.33
IT	60000.00
Finance	65000.00
Marketing	70000.00

• Q14. Find departments with more than 2 employees

Problem :
Write a query to find departments that have more than 2 employees.

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

Use HAVING to filter on aggregated results !

Result (sample) :

department	total_emp
HR	3
IT	4

• Q15. Find departments with total salary greater than 150000

Problem :
Write a query to find departments whose total salary is greater than 150000.

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 150000;
```

HAVING works like WHERE but for aggregated values !

Result (sample) :

department	total_salary
HR	175000
IT	240000



Key Takeaways

- ✓ GROUP BY groups rows with the same value.
- ✓ Use aggregate functions like COUNT(), SUM(), AVG(), MAX(), MIN().
- ✓ HAVING is used to filter grouped results.
- ✓ WHERE filters rows before grouping, HAVING filters after grouping.
- ✓ You can use multiple columns in GROUP BY.
- ✓ Aggregate functions ignore NULL values (except COUNT(*)).

"Group the data. Gain insights. Make better decisions !"

Practice SQL
Build Real Skills !

@codedsoul_05



Same
Concepts
Bigger
Opportunities !

SQL
Step by Step
Stronger
Foundations !

4. Joins (Q16 - Q20)

@codedsoul_05

Joins
Connect Data
Find Relationships
Like a Pro ! ☺

• Q16. INNER JOIN - Get matching records

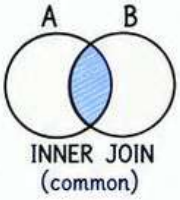
Problem :
Write a query to get employee names and their department names using INNER JOIN.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id;
```

INNER JOIN returns only matching records.

Result (sample) :

name	department_name
John	IT
Emma	HR
Mike	Finance



• Q17. LEFT JOIN - Get all employees with department (if available)

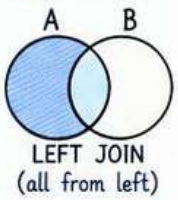
Problem :
Write a query to get all employees and their department names, including employees without a department.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id;
```

LEFT JOIN returns all records from left table, even if no match.

Result (sample) :

name	department_name
John	IT
Emma	HR
Mike	Finance
Sarah	NULL



• Q18. RIGHT JOIN - Get all departments with employees (if available)

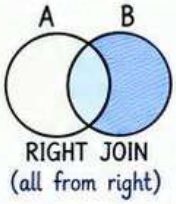
Problem :
Write a query to get all departments and their employees, including departments with no employees.

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.id;
```

RIGHT JOIN returns all records from right table, even if no match.

Result (sample) :

name	department_name
John	IT
Emma	HR
NULL	Marketing



• Q19. FULL OUTER JOIN - Get all records from both tables

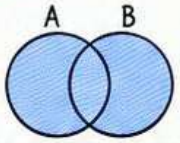
Problem :
Write a query to get all employees and all departments, including unmatched records from both tables.

```
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
ON e.department_id = d.id;
```

FULL OUTER JOIN returns all records from both tables, with NULLs where there is no match.

Result (sample) :

name	department_name
John	IT
Emma	HR
Mike	Finance
Sarah	NULL
NULL	Marketing



• Q20. SELF JOIN - Find employee and their manager

Problem :
Write a query to get each employee and their manager's name (from the same employees table).

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.id;
```

SELF JOIN joins a table with itself (using aliases).

Result (sample) :

employee	manager
John	NULL
Emma	John
Mike	John
Sarah	Emma

Key Takeaways

- ✓ INNER JOIN - returns only matching records.
- ✓ LEFT JOIN - returns all records from left table.
- ✓ RIGHT JOIN - returns all records from right table.
- ✓ FULL OUTER JOIN - returns all records from both tables.
- ✓ SELF JOIN - join a table with itself.
- ✓ Use JOINs to combine data from multiple tables.
- ✓ Useful in real-world interview and project scenarios.

" Good queries don't just fetch data,
they bring the right data together ! "

Practice More
Write Better SQL ! ☺

@codedsoul_05

Same Concepts
Bigger Opportunities ! ☺

SQL
Step by Step
Stronger
Foundations !

5. Subqueries (Q21 - Q25)

@codedsoul_05

Subqueries
Break Big Problems
Into Smaller
Queries !

Q21. Find employees with salary greater than the company average

Problem :
Write a query to find employees whose salary is greater than the average salary of all employees.

```
SELECT name, salary
FROM employees
WHERE salary > (
      SELECT AVG(salary)
      FROM employees
);
```

Subquery returns a single value (average salary).

Result (sample) :

name	salary
Mike	70000
Sarah	65000

Q22. Find employees from the IT department

Problem :
Write a query to find employees who work in the department 'IT' using a subquery.

```
SELECT name, salary
FROM employees
WHERE department_id IN (
      SELECT id
      FROM departments
      WHERE department_name = 'IT'
);
```

Subquery returns a list of values used with IN.

Result (sample) :

name	salary	department_id
John	60000	1
David	68000	1

Q23. Find departments with above average salary employees

Problem :
Write a query to find department names that have at least one employee with salary greater than the company average.

```
SELECT DISTINCT department_name
FROM departments
WHERE id IN (
      SELECT department_id
      FROM employees
      WHERE salary > (
            SELECT AVG(salary)
            FROM employees
      )
);
```

Nested subquery (subquery inside a subquery).

Result (sample) :

department_name
IT
Finance

Q24. Find the second highest salary

Problem :
Write a query to find the second highest salary from the employees table.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
      SELECT MAX(salary)
      FROM employees
);
```

Subquery gets the highest salary, outer query finds the next highest one.

Result (sample) :

second_highest
65000

Q25. Find employees who earn more than their department's average salary

Problem :
Write a query to find employees whose salary is greater than the average salary of their own department.

```
SELECT e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
      SELECT AVG(salary)
      FROM employees
      WHERE department_id = e.department_id
);
```

Correlated subquery (references outer query using alias).

Result (sample) :

name	salary	department_id
Mike	70000	3
Sarah	65000	2

Key Takeaways

- ✓ A subquery is a query inside another query.
- ✓ Subqueries can return a single value, a list of values, or a table.
- ✓ Use IN, NOT IN, EXISTS with subqueries.
- ✓ Correlated subqueries reference columns from the outer query.
- ✓ Subqueries help break complex problems into smaller steps.
- ✓ Common in interviews and real-world queries.

" Smaller queries, bigger insights ! "

Keep Practicing
Keep Growing !

@codedsoul_05

Query
Learn
Apply
Succeed !

SQL Step by Step Stronger Foundations !

6. CTEs + CASE (Q26 - Q30)

@codedsoul_05

CTEs for Readable Queries CASE for Smarter Logic !

Q26. Use CTE to find department-wise total salary

Problem :
Write a query using CTE to calculate the total salary for each department.

```sql
WITH dept_salary AS (
    SELECT department_id, SUM(salary) AS total_salary
    FROM employees
    GROUP BY department_id
)
SELECT department_id, total_salary
FROM dept_salary;
```

CTE makes complex queries more readable and reusable.

Result (sample) :

department_id	total_salary
1	120000
2	135000
3	70000

Q27. Use CTE to find highest paid employee in each department

Problem :
Write a query using CTE to find the highest paid employee in each department.

```sql
WITH ranked_employees AS (
    SELECT id, name, department_id, salary,
        ROW_NUMBER() OVER (PARTITION BY department_id
        ORDER BY salary DESC) AS rn
    FROM employees
)
SELECT id, name, department_id, salary
FROM ranked_employees
WHERE rn = 1;
```

CTE + Window Function to get top record per group.

Result (sample) :

id	name	department_id	salary
1	John	1	70000
4	Sarah	2	75000
6	David	3	50000

Q28. Categorize employees based on salary using CASE

Problem :
Write a query to categorize employees into salary ranges using CASE.

```sql
SELECT id, name, salary,
  CASE
    WHEN salary < 50000 THEN 'Low'
    WHEN salary BETWEEN 50000 AND 100000 THEN 'Medium'
    ELSE 'High'
  END AS salary_category
FROM employees;
```

CASE adds conditional logic in queries.

Result (sample) :

id	name	salary	salary_category
1	John	45000	Low
2	Emma	60000	Medium
3	Mike	120000	High

Q29. Count employees in each category using CASE

Problem :
Write a query to count the number of employees in each salary category.

```sql
SELECT
    CASE
      WHEN salary < 50000 THEN 'Low'
      WHEN salary BETWEEN 50000 AND 100000 THEN 'Medium'
      ELSE 'High'
    END AS salary_category,
    COUNT(*) AS employee_count
FROM employees
GROUP BY
    CASE
      WHEN salary < 50000 THEN 'Low'
      WHEN salary BETWEEN 50000 AND 100000 THEN 'Medium'
      ELSE 'High'
    END;
```

Use CASE in GROUP BY to aggregate dynamic categories.

Result (sample) :

salary_category	employee_count
Low	2
Medium	3
High	1

Q30. Use CTE and CASE to find department performance

Problem :
Write a query using CTE and CASE to classify departments based on their average salary.

```sql
WITH dept_avg AS (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
)
SELECT department_id, avg_salary,
    CASE
      WHEN avg_salary >= 100000 THEN 'High'
      WHEN avg_salary >= 60000 THEN 'Medium'
      ELSE 'Low'
    END AS performance
FROM dept_avg;
```

Combines CTE (for aggregation) with CASE (for classification).

Result (sample) :

department_id	avg_salary	performance
1	85000	Medium
2	120000	High
3	40000	Low

Key Takeaways

- ✓ CTEs (Common Table Expressions) make queries more readable and modular.
- ✓ CASE is used for conditional logic in queries.
- ✓ You can use CASE in SELECT, WHERE, ORDER BY and GROUP BY.
- ✓ CTEs help in breaking complex problems into smaller steps.
- ✓ Very commonly asked in SQL interviews and real-world analytics.

"Write smarter queries, not harder queries !"

Practice More Write Better SQL !

@codedsoul_05

Learn Practice Grow !

SQL
Step by Step
Stronger
Foundations !

7. Window Functions (Q31 - Q35)

@codedsoul_05

Window Functions
See Beyond Rows
Real Power
in SQL !

• Q31. Assign row number to employees based on salary (highest first)

Problem :
Write a query to assign a row number to each employee based on salary in descending order.

```
SELECT id, name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM employees;
```

ROW_NUMBER() assigns unique sequential numbers.

Result (sample) :

id	name	salary	rn
3	Mike	80000	1
1	John	60000	2
2	Emma	55000	3
4	Sarah	45000	4

• Q32. Find the rank of employees based on salary

Problem :
Write a query to find the rank of each employee based on salary in descending order.

```
SELECT id, name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

RANK() gives same rank for ties (skips next rank).

Result (sample) :

id	name	salary	rnk
3	Mike	80000	1
1	John	60000	2
2	Emma	60000	2
4	Sarah	45000	4

• Q33. Find the department-wise average salary using window function

Problem :
Write a query to show each employee's salary along with the average salary of their department.

```
SELECT id, name, department, salary,
       AVG(salary) OVER (PARTITION BY department)
       AS dept_avg
FROM employees;
```

PARTITION BY groups rows without collapsing them.

Result (sample) :

id	name	department	salary	dept_avg
1	John	IT	60000	65000
2	Emma	IT	70000	65000
3	Mike	HR	50000	55000
4	Sarah	HR	60000	55000

• Q34. Find the previous and next salary for each employee

Problem :
Write a query to show each employee's salary along with the previous and next higher salary.

```
SELECT id, name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

LAG() gets previous row and LEAD() gets next row.

Result (sample) :

id	name	salary	prev_salary	next_salary
4	Sarah	45000	NULL	50000
3	Mike	50000	45000	60000
1	John	60000	50000	70000
2	Emma	70000	60000	NULL

• Q35. Find the cumulative (running) total of salaries ordered by id

Problem :
Write a query to calculate the running total of salaries based on employee id.

```
SELECT id, name, salary,
       SUM(salary) OVER (ORDER BY id) AS running_total
FROM employees;
```

SUM() OVER creates a running total (cumulative sum).

Result (sample) :

id	name	salary	running_total
1	John	60000	60000
2	Emma	55000	115000
3	Mike	80000	195000
4	Sarah	45000	240000

Key Takeaways

- ✓ Window functions perform calculations across a set of rows related to the current row.
- ✓ Common functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM(), AVG(), MIN(), MAX().
- ✓ Use PARTITION BY to work on groups of rows.
- ✓ Window functions do not reduce the number of rows (unlike GROUP BY).
- ✓ Very useful for rankings, running totals, moving averages and comparative analysis.

"Window functions give you a new perspective on your data."

Practice Queries
Build Confidence !

@codedsoul_05

Practice
Apply
Grow
Succeed !

SQL Step by Step Stronger Foundations !

8. Date + String Queries (Q36 - Q40)

@codedsoul_05

Work with Dates
Manipulate Strings
Solve Real Problems !

• Q36. Find employees who joined in a specific year

Problem :
Write a query to find employees who joined in the year 2023 (from a column joining_date).

```
SELECT id, name, joining_date
FROM employees
WHERE YEAR(joining_date) = 2023;
```

Use YEAR() to extract year from date.

Result (sample) :

id	name	joining_date
1	John	2023-03-15
4	Sarah	2023-11-20

• Q37. Find employees who joined in the last 6 months

Problem :
Write a query to find employees who joined within the last 6 months from today.

```
SELECT id, name, joining_date
FROM employees
WHERE joining_date >= DATE_SUB(CURDATE(),
      INTERVAL 6 MONTH);
```

Use DATE_SUB() with INTERVAL to work with dates.

Result (sample) :

id	name	joining_date
2	Emma	2024-03-10
5	David	2024-06-01

• Q38. Find the number of employees joined each month

Problem :
Write a query to find how many employees joined in each month (for all months).

```
SELECT DATE_FORMAT(joining_date, '%Y-%m') AS month,
       COUNT(*) AS total_employees
FROM employees
GROUP BY DATE_FORMAT(joining_date, '%Y-%m')
ORDER BY month;
```

Use DATE_FORMAT() to group by month.

Result (sample) :

month	total_employees
2023-01	2
2023-03	1
2023-11	1
2024-01	1

• Q39. Find employees whose name starts with a specific letter

Problem :
Write a query to find employees whose name starts with the letter 'A'.

```
SELECT id, name
FROM employees
WHERE name LIKE 'A%';
```

Use LIKE with % for pattern matching.

Result (sample) :

id	name
6	Alex
8	Anna

• Q40. Extract parts of a string (first name and last name)

Problem :
Write a query to extract the first name and last name from a full_name column (assuming names are in format 'First Last').

```
SELECT full_name,
       SUBSTRING_INDEX(full_name, ' ', 1) AS first_name,
       SUBSTRING_INDEX(full_name, ' ', -1) AS last_name
FROM employees;
```

Use SUBSTRING_INDEX() to split strings.

Result (sample) :

full_name	first_name	last_name
John Doe	John	Doe
Emma Watson	Emma	Watson
Mike Ross	Mike	Ross

Key Takeaways

- ✓ Use YEAR(), MONTH(), DAY(), DATE_FORMAT() for date operations.
- ✓ Use DATE_SUB(), DATE_ADD(), CURDATE(), NOW() for relative dates.
- ✓ Use LIKE, LEFT(), RIGHT(), SUBSTRING(), SUBSTRING_INDEX() for string operations.
- ✓ String functions are case-sensitive (depending on collation).
- ✓ Date + String queries are very common in real-world projects and interviews.

" Data tells a story, strings give it meaning ! "

Practice Queries
Build Problem Solving !

@codedsoul_05

Same Concepts
Bigger Opportunities !

SQL Step by Step Stronger Foundations !

9. Advanced SQL Interview Queries (Q41 - Q45)

Think Deeper Write Smarter Crack Interviews !

@codedsoul_05

Q41. Find the second highest salary from a table (without using LIMIT)

Problem :
Write a query to find the second highest salary from the employees table.

```sql
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
      SELECT MAX(salary)
      FROM employees
);
```

Find second highest without using LIMIT or TOP.

Result (sample) :

second_highest
70000

Q42. Find duplicate records based on a column (e.g., email)

Problem :
Write a query to find all duplicate emails and their count from the users table.

```sql
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Use GROUP BY with HAVING to find duplicates.

Result (sample) :

email	cnt
john@example.com	2
emma@example.com	3

Q43. Find the Nth highest salary (e.g., 3rd highest)

Problem :
Write a query to find the 3rd highest salary from the employees table.

```sql
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
OFFSET 2 ROWS FETCH NEXT 1 ROW ONLY;
```

Use OFFSET FETCH (or DENSE_RANK()) for Nth highest.

Result (sample) :

salary
60000

Q44. Find employees who have not placed any orders (using NOT EXISTS)

Problem :
Write a query to find all customers who have not placed any orders.

```sql
SELECT c.id, c.name
FROM customers c
WHERE NOT EXISTS (
      SELECT 1
      FROM orders o
      WHERE o.customer_id = c.id
);
```

Use NOT EXISTS to find records that do not have matching records.

Result (sample) :

id	name
3	Mike
5	Sarah

Q45. Find the top 3 departments by total salary (using window function)

Problem :
Write a query to find the top 3 departments with the highest total salary.

```sql
SELECT department, total_salary, rn
FROM (
    SELECT department,
        SUM(salary) AS total_salary,
        ROW_NUMBER() OVER (ORDER BY SUM(salary) DESC) AS rn
    FROM employees
    GROUP BY department
) t
WHERE rn <= 3;
```

Use window function (ROW_NUMBER()) for top N results.

Result (sample) :

department	total_salary	rn
IT	250000	1
HR	180000	2
Finance	120000	3

Key Takeaways

- ✓ Use subqueries, window functions and EXISTS / NOT EXISTS for advanced problems.
- ✓ Be comfortable with duplicate handling, ranking, top N queries and missing data queries.
- ✓ Window functions (ROW_NUMBER(), RANK(), DENSE_RANK()) are very powerful.
- ✓ Think about performance and real-world use cases.
- ✓ These queries are commonly asked in SQL interviews and data engineering roles.

" Advanced queries turn good SQL developers into great ones ! "

Practice Advanced Think Like an Engineer !

@codedsoul_05

Better Queries Bigger Opportunities !

SQL
Step by Step
Stronger
Foundations !

10. Top 50 SQL Revision Cheat Sheet (Q46 - Q50)

Revise
Practice
Remember
Crack Interviews !

@codedsoul_05

Q46. Find the first and last login date for each user

Problem :
Write a query to get the first and last login date for each user from the logins table.

```sql
SELECT user_id,
       MIN(login_date) AS first_login,
       MAX(login_date) AS last_login
FROM logins
GROUP BY user_id;
```

Use MIN() and MAX() for first and last occurrence.

Result (sample) :

user_id	first_login	last_login
1	2023-01-10	2023-06-20
2	2023-02-15	2023-08-05
3	2023-03-01	2023-07-18

Q47. Find the running total of sales for each month

Problem :
Write a query to get monthly sales and the running total using a window function.

```sql
SELECT month,
       SUM(sales) AS monthly_sales,
       SUM(SUM(sales)) OVER (ORDER BY month) AS running_total
FROM sales
GROUP BY month
ORDER BY month;
```

Use window function with SUM() OVER().

Result (sample) :

month	monthly_sales	running_total
2023-01	1000	1000
2023-02	1500	2500
2023-03	2000	4500

Q48. Find the most frequent customer (highest orders)

Problem :
Write a query to find the customer who has placed the maximum number of orders.

```sql
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id
ORDER BY total_orders DESC
LIMIT 1;
```

Use GROUP BY with COUNT() and ORDER BY DESC with LIMIT 1.

Result (sample) :

customer_id	total_orders
5	25

Q49. Find employees who earn more than their department's average salary

Problem :
Write a query to find employees whose salary is greater than the average salary of their own department.

```sql
SELECT e.id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
        SELECT AVG(salary)
        FROM employees
        WHERE department_id = e.department_id
);
```

Use a correlated subquery.

Result (sample) :

id	name	salary	department_id
1	John	80000	1
4	Sarah	75000	2

Q50. Find the top 3 products by total revenue

Problem :
Write a query to find the top 3 products based on total revenue (quantity * price).

```sql
SELECT product_id, SUM(quantity * price) AS total_revenue
FROM order_items
GROUP BY product_id
ORDER BY total_revenue DESC
LIMIT 3;
```

Use aggregate expression and ORDER BY with LIMIT.

Result (sample) :

product_id	total_revenue
3	150000
1	120000
2	95000

Quick Tricks for SQL Interviews

1. Use LIMIT (or TOP) to get top N results.
2. Use DISTINCT to remove duplicates.
3. Use COALESCE(col, 0) to handle NULLs.
4. Use CASE for conditional logic.
5. Use window functions for rankings, running totals, etc.
6. Use EXISTS / NOT EXISTS for better performance (instead of IN / NOT IN).
7. Use CTEs to make complex queries readable.
8. Use meaningful aliases to improve readability.
9. Know date functions: CURRENT_DATE, DATE_ADD, DATE_SUB, EXTRACT.
10. Always check edge cases (NULLs, ties, empty results).

Remember :

- ✓ SQL is not just about writing queries, it's about solving real business problems.
- ✓ Practice these 50 questions regularly.
- ✓ Try variations of each question.
- ✓ Understand the logic, don't just memorize the syntax.
- ✓ With practice, you'll crack any SQL interview !

Revise SQL
Build Confidence !

Same Concepts
Bigger Opportunities !

Consistent Practice Today → Better Opportunities Tomorrow !